

Validação do sistema BioByte Sentinela — o que existe, o que funciona, o que falta

Feita contra o **sistema implantado** pelo LangNet (implantação 46ce5f01, geração 3e81772f), executando os **43 casos de teste** que a própria etapa de Casos de Teste gerou. Método e limites descritos ao final — inclusive onde o teste não conclui nada.

1. O que tem no sistema (menus e opções)

4 módulos, 20 telas:

Módulo	Telas
Atendimento	Login e MFA · Alerta MDR · Detalhe do Caso · Logs de Auditoria
Engajamento	Detalhe do Caso Clínico (visão de classificação)
Relatórios	Geração de Relatórios
Cadastros	Importação de Microbiologia · Prévia de Resultados · Recomendação de Bundle · Resultado da Estimativa de Risco · Dashboard de Vigilância · Gestão de Usuários · +8 cadastros: Alertas, Casos, Escores de Risco, Logs de Auditoria, Microbiologias, Pacientes, Tratamentos, Usuários

Observação de organização: telas de negócio (Dashboard, Recomendação de Bundle) estão classificadas no módulo "Cadastros", e o Detalhe do Caso Clínico em "Engajamento" — os módulos vêm da etapa de Interface e não refletem bem o domínio clínico. Não afeta função; afeta usabilidade.

2. Os cadastros necessários existem?

Sim, todos. 8 tabelas no modelo de dados, **8 cadastros com CRUD completo** (criar, listar, obter, atualizar, excluir): alertas, casos, escores_risco, logs_auditoria, microbiologias, pacientes, tratamentos, usuarios. **Nenhuma tabela sem cadastro na interface.**

3. Rastreabilidade — os requisitos e casos de uso foram implementados?

Na estrutura, cobertura total:

- **12 de 12 casos de uso** têm tarefa correspondente, com rastreabilidade declarada.
- **14 de 14 requisitos funcionais** aparecem ligados a alguma tarefa.

Caso de uso	Tarefa	Modo	Requisitos
UC-001 Autenticar com MFA	authenticate_user_mfa	fixo	FR-010
UC-002 Iniciar importação	init_import_session	fixo	FR-002
UC-003 Integração microbiologia	fetch_and_persist_microbiology	fixo	FR-002, FR-007
UC-004 Classificar NHSN	classify_case_nhsn	agente	FR-003
UC-005 Detectar MDR	detect_mdr_and_alert	fixo	FR-004, FR-013
UC-006 Escore de Cox	calculate_cox_risk_score	fixo	FR-001, FR-008
UC-007 Recomendar bundle	recommend_treatment_bundle	agente	FR-005
UC-008 Estimar redução de risco	estimate_risk_reduction	agente	FR-006
UC-009 Dashboard	generate_dashboard_metrics	fixo	FR-012
UC-010 Gerenciar usuários	manage_user_account	fixo	FR-010, FR-011
UC-011 Logs de auditoria	query_audit_logs	fixo	FR-011
UC-012 Exportar relatório	export_vigilance_report	fixo	FR-014
(integração)	orquestrar_fluxo_clinico_integrado	fixo	FR-009

Mas cobertura estrutural não é cumprimento. A execução dos casos de teste mostra onde a implementação fica devendo — seção 5.

4. Os agentes funcionam? Há respostas?

Sim, no caminho normal. A cadeia clínica completa roda no sistema implantado (**12 de 13 tarefas**; a 13ª é administração de usuário, que exige dados de usuário que a cadeia clínica não carrega). Respostas reais observadas:

- classificação NHSN com justificativa citando a regra BR-001 e os critérios do caso;
- escore de Cox **0,2437 (risco baixo)** com os fatores listados, vindo do serviço externo;
- recomendação **"Bundle MRSA — Vancomicina"** com justificativa clínica (72 anos, UTI, cateter central há 12 dias, nutrição parenteral);
- estimativa de redução de risco **−35%**, intervalo [−20, −50] — **ATENÇÃO: esse valor NÃO é calculado.** É o exemplo ilustrativo do wireframe da Especificação, que entrou na instrução da tarefa como "ex: -35" e o agente repete. Achado na auditoria de valores falsos (04/09/2026); 42 tratamentos no banco carregam exatamente esse número;
- microbiologia buscada no laboratório externo: *Staphylococcus aureus*, multirresistente, com antibiograma.

Monitoramento do sistema no ar: **13 tarefas exercitadas, 13 sucessos, 7,1 s de média, 3 chamadas a ferramentas externas.**

5. Execução dos 43 casos de teste — resultado honesto

Categoria	Qtd	O que significa
Comportamento verificado OK	3	executado no sistema e conferido
Comportamento com defeito confirmado	7	o sistema não faz o que o caso de uso exige
Não exercitável pelo meu método	3	exigem induzir falha interna (erro de regra, falha de carga)
Elemento de interface existe	18	conferido no código das telas geradas
Elemento de interface faltando	7	especificado no caso de uso, ausente na tela
Efeito não implementado	5	e-mail/push, timeout, processamento assíncrono, skeleton
Caso de uso sem casos gerados	1	UC-010 — a própria etapa de Casos de Teste falhou aqui

5.1 Defeitos confirmados (7)

1. **FR-014 — Exportação de relatórios NÃO gera arquivo.**
export_vigilance_report devolve os dados da consulta e nenhum PDF/CSV. **Causa raiz:** a tarefa está marcada como procedimento fixo, e procedimento fixo não chama ferramenta — embora o agente de relatórios tenha pdf_writer e csv_writer ligados e as ferramentas existam no sistema. É a mesma classe de erro de classificação já corrigida para "tarefa que precisa decidir um valor"; falta cobrir "tarefa que precisa produzir um artefato com ferramenta". (TC-UC-012-01, TC-UC-012-02)
2. **FR-011 — Consulta de logs devolve vazio.** query_audit_logs responde logs: null mesmo havendo registros, e não distingue "nenhum resultado". A gravação de auditoria funciona; a consulta, não. (TC-UC-011-02)
3. **Não valida entrada insuficiente — o sistema responde mesmo sem os dados.** Removendo os parâmetros clínicos obrigatórios, o sistema **calculou o score, recomendou bundle e estimou redução de risco assim mesmo**, em vez de indicar campos faltantes. Os casos de uso preveem "lista de campos obrigatórios faltantes", "nenhuma recomendação disponível", "estimativa não disponível". Em domínio clínico, responder sem dado é pior do que recusar. (TC-UC-006-02, TC-UC-007-02, TC-UC-008-02)
4. **Não bloqueia dados não conformes na importação.** Com um caso inexistente, a importação de microbiologia concluiu e gravou registro, em vez de bloquear. (TC-UC-003-04)

5.2 Elementos de interface especificados que não existem (7)

- badges de classificação "**ICSAC Confirmado**" / "**Não ICSAC**" / "**Classificação Pendente**" (UC-004);

- mensagens "**Credenciais inválidas**" e "**Código incorreto**" no login (UC-001);
- botão "**Exportar CSV**" nos logs de auditoria (UC-011);
- **barra de progresso** do escore de risco (UC-006).

5.3 Efeitos não implementados (5)

- **Notificação por e-mail e push do alerta MDR** (FR-013 fica parcial: grava o alerta, não avisa);
- tratamento de **timeout** com mensagem e log;
- **processamento assíncrono** para exportação de grande volume;
- **skeleton loader** durante carga lenta.

6. Limites deste teste (o que ele NÃO prova)

- Os casos de teste são de causa-efeito no nível de comportamento e de tela. Só o comportamento foi **executado** no sistema; os elementos de interface foram conferidos **no código das telas**, não clicando — a existência do texto/campo não garante que apareça na hora certa.
- **3 casos não são exercitáveis** pelo método: exigem induzir falha interna (erro na regra de classificação, falha ao carregar recomendações, erro no cálculo). Não são falhas comprovadas do sistema, e também não são aprovações.
- Casos negativos foram construídos removendo/invalidando campos. Como algumas tarefas completam dados a partir do banco, parte do "respondeu mesmo sem dado" pode vir daí — o que **não** anula o achado: o caso de uso pede validação explícita de entrada, que não existe.
- Este relatório mede o **sistema gerado**, não o gerador. As causas raiz apontadas (classificação de tarefa, ausência de validação) é que devem virar correção no LangNet.

7. Conclusão direta

O sistema **existe de verdade**: cadastros completos, 12 de 12 casos de uso com tarefa, agentes respondendo com conteúdo clínico coerente, cadeia completa rodando no ambiente implantado.

Mas **o caminho de exceção está em falta**: o sistema quase não valida entrada, não gera os arquivos de relatório, não notifica, e sete elementos de tela previstos nos casos de uso não existem. Em outras palavras: **o feliz caminho foi implementado; os requisitos de robustez e os artefatos de saída, não.**

Reexecutar: `python3 e2e/run_test_cases.py <porta_do_servidor_de_agentes>`

7. Segunda rodada: os defeitos corrigidos NO GERADOR e reconferidos

Cada defeito da seção 5 foi corrigido no **gerador** (nunca no aplicativo) e reconferido do jeito mais caro: **regerando o sistema inteiro pela interface do LangNet e implantando de novo**, uma vez por correção. Foram **cinco regenerações e cinco implantações** nesta rodada, todas pela interface (sessões 332b0907, 58a867b8, d6b52738, 7b2e9e88, 388013c4; implantação final nas portas 5014 / 8008 / 3010).

7.1 Placar antes e depois

Categoria	Antes	Depois
Comportamento verificado OK	3	9
Comportamento com defeito confirmado	7	0
Não exercitável pelo método	3	6
Elemento de interface existe	18	25
Elemento de interface faltando	7	0
Efeito não implementado	5	5
Caso de uso sem casos gerados	1	1

O total de casos subiu de 43 para 46 verificações: os 43 casos gerados, mais uma **sonda de recusa** aplicada a duas tarefas (o agente inventa ou declara falta de dado?), mais o caso de uso sem casos. "Não exercitável" subiu de 3 para 6 porque três casos que antes eram reprovados por um critério grosseiro passaram a ser classificados com honestidade: a condição que eles exigem (a base de protocolos falhar, o perfil não ter estimativa) não se cria pela entrada.

7.2 O que foi corrigido, defeito a defeito

1. **Exportação não gerava arquivo.** A tarefa de exportação estava marcada como procedimento fixo, e procedimento fixo não chamava ferramenta. Agora, quando a tarefa produz um artefato, o sistema aciona a ferramenta de PDF ou CSV depois de gravar. Prova: a resposta traz o caminho do arquivo, em PDF e em CSV. (TC-UC-012-01, TC-UC-012-02 passam)
2. **Consulta de auditoria devolvia vazio.** A consulta selecionava várias colunas e o gerador guardava uma única célula da primeira linha. Agora guarda o conjunto de linhas, que volta como lista vazia quando não há nada — e a tela diz "Nenhum registro encontrado para os filtros aplicados". (TC-UC-011-02 passa)
3. **Sistema respondia sem os dados obrigatórios.** Passou a existir pré-condição derivada do modelo de dados e dos parâmetros da ferramenta externa: sem os fatores clínicos, o cálculo do score recusa e lista o que falta. (TC-UC-006-02 passa)

4. **O agente inventava quando faltava dado.** Duas mudanças: toda tarefa de agente recebe a regra de dados insuficientes, e o contrato de saída passou a aceitar a recusa como resposta válida. Faltava uma terceira: quando a resposta não cumpria o contrato, o servidor **recobrava os campos obrigatórios**, empurrando o agente a inventar. Agora a recobrança oferece a saída honesta. Prova: com um caso sem dado nenhum, as duas tarefas responderam "dados insuficientes" com o motivo. (sonda de recusa passa nas duas tarefas)
5. **Importação aceitava dado fora do padrão.** O sistema já bloqueava, mas com jargão interno. Agora responde "dados não conformes recebidos do sistema externo — importação bloqueada", dizendo quais campos faltaram. (TC-UC-003-04 passa)
6. **Sete elementos de interface especificados não existiam.** Causa raiz: a especificação diz o que cada desfecho deve exibir — selo "ICSAC Confirmado", "Não ICSAC", "Classificação Pendente", as mensagens "Credenciais inválidas" e "Código incorreto", o botão "Exportar CSV", a barra do score — e **nada disso chegava ao gerador de código**. Agora a especificação funcional é carregada junto com os demais artefatos, o vocabulário de cada caso de uso é extraído e a tela passa a renderizar o desfecho com ele: valor especificado vira selo colorido, score vira barra de progresso, lista de registros ganha exportação. (os 7 casos de interface passam)
7. **Jargão interno vazando para o usuário.** O login com senha errada respondia "verificação (pós) falhou: output_has:usuario_id". Duas correções: a consulta que não acha o registro **para a tarefa** em vez de seguir gravando nulo, e o servidor traduz a falha para a mensagem do caso de uso. Na tela, agora, "Credenciais inválidas" — com o detalhe técnico guardado à parte.

7.3 O que continua fora (sem maquiagem)

- **5 efeitos não implementados:** e-mail e push do alerta MDR, tratamento de tempo esgotado, processamento assíncrono da exportação, carregamento animado do painel.
- **6 casos não exercitáveis** por fora: exigem provocar falha interna (erro da regra de classificação, base de protocolos indisponível, perfil sem estimativa, erro no cálculo, reenvio de código após falha).
- **1 caso de uso sem casos de teste:** gestão de usuários e permissões — a falha é da etapa que gera os casos, não do sistema.

7.4 Um defeito que o próprio teste criou (e como foi pego)

A primeira versão da correção da consulta de auditoria era larga demais: qualquer consulta com várias colunas virava lista. O login guardava o identificador do usuário a partir de uma consulta com quatro colunas e passou a receber uma lista onde esperava um valor — o registro de auditoria quebrou com "lista não pode virar tipo do banco". A regra foi estreitada para valer só quando o nome guardado é de coleção. Fica o registro: a correção entrou, quebrou outra coisa, a bateria pegou, e a bateria foi rodada de novo até fechar em zero falhas.

8. Auditoria de valores falsos (04/09/2026)

Varredura do app gerado à procura de qualquer coisa simulada. Resultado: **13 implementações de mentira**, quatro delas no caminho que o sistema usa.

Achado	Situação
Login não conferia senha nem código de verificação	CORRIGIDO — senha errada agora devolve "Credenciais inválidas"; senha certa entra
Detector de multirresistência alertava em toda amostra	CORRIGIDO — amostra com 2 resistências dá <code>is_mdr: false</code> e não gera alerta
Estimativa de redução de risco repete o exemplo da Especificação (−35%)	ABERTO — não há modelo estatístico; o exemplo vai na instrução e o agente copia
Relatório de vigilância gera arquivo sem os dados da vigilância	ABERTO — a consulta traz 8 colunas e o gerador guarda uma célula
9 ferramentas com corpo inventado (token "fake_jwt_token", validador de senha sempre verdadeiro)	CORRIGIDO na origem — a etapa Ferramentas manda no código; sem implementação declarada, a ferramenta falha explícito
Laboratório externo com dois pacientes escritos no código	POR DESENHO — é a fronteira do sistema; um laboratório real se pluga no mesmo ponto
Coeficientes do modelo de Cox "ilustrativos"	POR DESENHO — a fórmula é real e determinística; os coeficientes não vêm de estudo publicado

Causa estrutural: passos de LÓGICA da descrição da tarefa (conferir hash, contar resistências, condicionar o alerta) não viravam SQL e sumiam sem aviso. O tradutor passou a emití-los e a **listar no log da geração** o que ainda não converte — 23 passos no BioByte.